

1. Protocols and the layered model

A protocol is an agreed set of rules for formatting and exchanging messages between machines. Real networks stack several protocols on top of one another, each solving one part of the problem. This primer uses the four-layer TCP/IP model: the link layer moves frames between two directly connected machines; the internet layer moves packets across networks using IP addresses; the transport layer gives applications either a reliable ordered byte stream (TCP) or lightweight best-effort datagrams (UDP); and the application layer is where protocols such as HTTP and DNS live.

Each layer wraps the data handed down from the layer above it in its own header, a process called encapsulation. As a packet travels down the sending stack it gains a transport header, then an IP header, then a link-layer frame header; the receiving stack strips those headers off in the reverse order on the way back up. A layer only ever reads its own header, which is what lets one layer be changed without disturbing the others.

2. The link layer

The link layer delivers frames between machines on the same physical or logical network segment. Every network interface has a MAC address: a 48-bit identifier, usually written as six hex bytes, that is assigned by the manufacturer and is meant to be globally unique.

An Ethernet frame carries a destination MAC, a source MAC, a type field, the payload handed down from the internet layer, and a trailing checksum. A switch forwards frames by learning which MAC address sits on which port: it inspects the source address of every frame it receives, records the port it arrived on, and from then on sends frames for that destination out only that one port. Frames for an address it has not yet learned, and broadcast frames, are flooded out every port except the one they came in on. All the interfaces a broadcast frame can reach form one broadcast domain.

3. The internet layer and IP addressing

The internet layer moves packets between networks. An IPv4 address is 32 bits, written as four decimal octets separated by dots, for example 203.0.113.7. Three ranges are reserved for private use and are never routed on the public internet: 10.0.0.0/8, 172.16.0.0/12, and 192.168.0.0/16. Hosts on a private network reach the internet through network address translation at their gateway.

IPv4's roughly four billion addresses proved too few, so IPv6 was introduced with 128-bit addresses, written as eight groups of four hex digits with runs of zeros collapsed to a double colon. IPv6 also removes the need for NAT in most deployments because every host can have a globally unique address.

4. Subnets and CIDR notation

A subnet is a contiguous block of IP addresses sharing a common prefix. CIDR notation writes the prefix length after a slash: in 192.168.1.0/24 the first 24 bits are the network portion and the remaining 8 bits identify a host within it.

A /24 therefore spans 256 addresses, but two are not usable as host addresses: the all-zeros host part is the network address that names the subnet itself, and the all-ones host part is the broadcast address for that subnet. That leaves 254 usable host addresses in a /24. Every bit added to the prefix halves the block: a /25 is 128 addresses, a /26 is 64, and so on. Routers and hosts decide whether a destination is 'on my subnet' or 'must go through the router' by comparing prefixes.

5. Routing and the path across the internet

A router connects two or more networks and forwards packets between them. It keeps a routing table of destination prefixes, each paired with a next hop. When a packet arrives the router picks the table entry whose prefix matches the destination address and is the most specific, that is, has the longest prefix length. This is longest-prefix match. If nothing else matches, the default route 0.0.0.0/0 is used; on a host that next hop is the default gateway.

Every IP packet carries a time-to-live field. Each router that forwards the packet decrements the TTL by one, and if the TTL reaches zero the packet is discarded and the router sends an ICMP time-exceeded message back to the source. This both stops packets from circulating forever in a routing loop and is the mechanism the traceroute tool exploits to map a path hop by hop.

6. The transport layer

Two transport protocols sit on top of IP. TCP provides a connection-oriented, reliable, ordered byte stream: bytes handed to the sender arrive once, in order, or the connection fails. UDP provides connectionless best-effort datagrams with no delivery, ordering, or duplicate-suppression guarantees; it is used where low overhead matters more than reliability, such as DNS queries and live media.

Both use 16-bit port numbers to identify which application a segment belongs to, so a single host can hold many connections at once. Ports below 1024 are the well-known ports reserved for standard services. A TCP connection opens with a three-way handshake: the client sends a SYN, the server replies with a SYN-ACK, and the client answers with an ACK, after which data can flow.

7. How TCP stays reliable

TCP numbers every byte it sends. Each segment carries a sequence number identifying its first byte, and each acknowledgement carries the number of the next byte the receiver expects, which implicitly confirms everything before it. If an acknowledgement does not arrive before a retransmission timer expires, the sender resends the unacknowledged data.

Flow control stops a fast sender from overwhelming a slow receiver: the receiver advertises a window saying how many more bytes it can currently buffer, and the sender never has more than that outstanding. Congestion control, which is separate, stops senders from overwhelming the network: a connection starts with a small congestion window and grows it as acknowledgements return, backing off sharply when loss is detected. A connection is closed with a FIN from each side, each FIN being acknowledged.

8. The Domain Name System

DNS translates human-readable names such as `example.com` into IP addresses. The namespace is a hierarchy: the root servers delegate to top-level domain servers such as `.com`, which delegate to the authoritative servers for an individual domain. A client normally asks a recursive resolver, which walks that hierarchy on the client's behalf and returns the final answer.

Common record types: an A record maps a name to an IPv4 address; an AAAA record maps a name to an IPv6 address; a CNAME record makes one name an alias for another; an MX record names the mail servers for a domain; NS records name a zone's authoritative servers; and a TXT record holds arbitrary text. Every record carries a time-to-live that tells resolvers how many seconds they may cache it before asking again.

9. HTTP

HTTP is the request/response protocol of the web. A request has a method, a path, headers, and an optional body. GET retrieves a resource, POST submits data, PUT replaces a resource, DELETE removes one, and HEAD asks for the headers a GET would return without the body.

Every response carries a three-digit status code. The first digit gives the class: 1xx informational, 2xx success, 3xx redirection, 4xx client error (the request was malformed or not allowed, as with 404), and 5xx server error (the server failed to fulfil a valid request). HTTP is stateless, so servers use cookies to recognise repeat visitors. HTTP/1.1 keep-alive reuses one TCP connection for several sequential requests; HTTP/2 goes further and multiplexes many concurrent requests and responses over a single connection.

10. TLS and a full page load

TLS wraps an ordinary TCP connection to give it three properties: confidentiality, so an eavesdropper learns nothing; integrity, so tampering is detected; and authentication, so the client knows which server it is talking to. The handshake uses asymmetric cryptography, together with the server's certificate, only to authenticate the server and agree on a shared symmetric session key; the much faster symmetric key then encrypts the actual application data. A certificate is trusted because it is signed by a certificate authority whose own certificate chains up to one already trusted by the client.

Putting it together, loading `https://example.com/` means: resolve the name via DNS; open a TCP connection with the three-way handshake; run the TLS handshake over it; send an HTTP GET for `/`; receive a 200 response whose body is HTML; and repeat the process for each stylesheet, script, and image the page references.